A Project Report on

Stock Watchlist Assistant with Explainable Signal Fusion for Retail Investors

Field	Value
Subject	AI in Fintech
Programme	MTech AI/ML
Institution	Symbiosis Institute of Technology, Pune
Academic Year	2025-26
GitHub	https://github.com/tp-0604/stock-watchlist
FinBERT Model	huggingface.co/tahp0604/finbert-sentfin

This document is a combined project report and application reference guide. It covers the motivation, technical design, complete setup instructions, and usage guide for running the stock watchlist application on any machine.

Submitted by
Taher Poonawala 25070149025

Submitted To:
Dr. Anupkumar M Bongale

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Pune – 412115, Maharashtra State, India
https://www.sitpune.edu.in/

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
Academic Year: 2025 – 2026

Table of Contents

1. Statement of Need
- 1.1 The Retail Investor Problem
- 1.2 Limitations of Existing Tools
- 1.3 Core Need Statement

2. Setup and Installation
- 2.1 Prerequisites
- 2.2 Get API Key (Free)
- 2.3 Option A: Docker (Recommended)
- 2.4 Option B: Linux / macOS (venv)
- 2.5 Option C: Windows
- 2.6 FinBERT Model (Automatic Download)

3. Technical Functionality
- 3.1 Pipeline Overview
- 3.2 Technical Indicator Scoring
- 3.3 News Sentiment Scoring
- 3.4 Explainable Signal Fusion (ESF)
- 3.5 LLM Digest Generation

4. Architecture
- 4.1 System Architecture Diagram
- 4.2 Two-Path Design
- 4.3 Project Structure
- 4.4 API Endpoints
- 4.5 Configuration Reference

5. Usage and Scope
- 5.1 Adding Stocks to Your Watchlist
- 5.2 Running Analysis
- 5.3 Reading a Card
- 5.4 14-Day History Panel
- 5.5 Scope and Limitations

6. Impact Overview Statement
- 6.1 Research Contribution: Explainable Signal Fusion
- 6.2 Domain Contribution: Fine-tuned FinBERT
- 6.3 Impact Summary Metrics
- 6.4 Closing Statement

7. Troubleshooting
- 7.1 Verifying Raw Indicator Values

References

1. Statement of Need

1.1 The Retail Investor Problem

During the fiscal year 2023-24, India experienced a surge in the opening of demat accounts of about 30 million people, elevating the total number of registered retail investors in the country to more than 150 million. Most of these new investors have no prior experience or formal education in financial market analysis. They rely heavily on social media posts, word of mouth, or broker's suggestions for their investment decisions that become a random choice to them as they do not have systematic ways of identifying whether a stock is a suitable entry or exit point. The problem does not stem from a lack of data. Stock price data, technical indicators, and financial news are all available at no cost. The issue is synthesis: no user-friendly tool exists that takes all these signals, settles conflicts between them, and then explains the outcome in clear English linked to a particular personal objective.

1.2 Limitations of Existing Tools

- Traditional charting tools (TradingView, Zerodha Kite) display RSI, MACD, and Bollinger Bands visually. The user must interpret these signals themselves — requiring financial literacy that most retail investors do not have.
- LLM-based research systems (TradingAgents, MarketSenseAI, FinAgent, Ploutos) are designed for institutional or algorithmic use. They require significant API costs, target quantitative researchers, and produce no personalised goal-aware daily digest for a non-technical user.

A literature review of 36+ published systems confirmed that no existing work combines: (a) live technical indicators, (b) Indian financial news sentiment from a domain-adapted model, (c) explicit conflict detection before any LLM involvement, and (d) a goal-aware plain-English digest in a single, free, reproducible system targeting retail investors.

System	Year	Gap vs This project
TradingAgents	2024	LLM-implicit fusion, institutional target, requires GPT-4
MarketSenseAI 2.0	2025	No TA indicators, no conflict detection, no retail UI
FinAgent	2024	Neural fusion, no explainability, no daily digest
Ploutos	2024	Binary prediction, no goal-awareness, no conflict detection
SEP (Koa et al.)	2024	No TA indicators, no Indian market focus

1.3 Core Need Statement

Core Need: There is a need for a lightweight, cost-free AI system that continuously monitors a personalised stock watchlist, detects conflicts between technical and sentiment signals before any LLM is involved, and delivers a once-daily plain-English digest aligned to each user's specific investment goal, without requiring any financial expertise to use.

2. Setup and Installation

2.1 Prerequisites

Requirement	Version	Notes
Python	3.10 or higher	python.org — tick 'Add to PATH' on Windows
Git	Any recent	To clone the repository
Docker Desktop	Optional	Only for Docker deployment option

2.2 Get API Key (Free)

Key	Where	Steps
GROQ_API_KEY	console.groq.com	Sign up free → API Keys → Create key. No credit card required.

2.3 Option A: Docker (Recommended, Any OS)

Requires only Docker Desktop: no Python installation needed. The fine-tuned FinBERT model is downloaded and built into the image at build time.

```
git clone https://github.com/tp-0604/stock-watchlist.git
cd stock-watchlist
cp .env.example .env          # add GROQ_API_KEY to this file
docker compose up --build
```

First build downloads tahp0604/finbert-sentfin (~440 MB) and builds it into the image. All subsequent docker compose up commands start instantly with no re-download.

Open `http://localhost:5000` in your browser to use the app. To stop: `docker compose down`.

2.4 Option B: Linux / macOS (venv)

```
git clone https://github.com/tp-0604/stock-watchlist.git
cd stock-watchlist
bash setup.sh
# Add GROQ_API_KEY to .env, then:
bash run.sh
```

`setup.sh` automatically detects Python 3.10+, creates `venv/`, installs all dependencies, downloads the fine-tuned FinBERT model once (~440 MB), and copies `.env.example` to `.env`.

Open `http://localhost:5000` in your browser to use the application.

2.5 Option C: Windows

Install Python 3.10+ from python.org and tick "Add Python to PATH" during install. Then from Command Prompt:

```
git clone https://github.com/tp-0604/stock-watchlist.git
cd stock-watchlist
setup.bat
# Open .env and add GROQ_API_KEY, then:
run.bat
```

`setup.bat` automatically detects Python 3.10+, creates `venv/`, installs all dependencies, downloads the fine-tuned FinBERT model once (~440 MB), and copies `.env.example` to `.env`.

Open `http://localhost:5000` in your browser to use the application.

2.6 FinBERT Model (Automatic Download)

`finbert-sentfin` (the fine-tined FinBert model) is hosted on HuggingFace Hub and loaded via `transformers.from_pretrained()`. No manual download is needed. Auto-detection logic in `src/config.py`: if `models/finbert-sentfin/` exists locally, the app uses it. Otherwise it downloads `tahp0604/finbert-sentfin` from HuggingFace automatically on first run.

3. Technical Functionality

3.1 Pipeline Overview

Every call to /analyse/<ticker> runs these steps sequentially:

#	Module	Input	Output
1	price_fetcher.py	Ticker symbol	OHLCV + RSI, MACD, BB values (PriceData)
2	news_fetcher.py	Ticker symbol	Up to 5 recent headlines (list[NewsItem])
3	sentiment_analyzer.py	Headlines	Weighted sentiment score + label (SentimentSummary)
4	technical_signals.py	PriceData	TA label per indicator + composite score (TechnicalSignal)
5	signal_fusion.py	TechnicalSignal + SentimentSummary	Fusion score, conflict flag, overall signal (FusionResult)
6	digest_generator.py	FusionResult + user goal	Plain-English paragraph (string)
7	db.py	DigestCard	Row saved to digest_history.db

3.2 Technical Indicator Scoring

Price data is fetched via `yfinance` using 1 year of daily OHLCV. `pandas-ta` computes three indicators:

Indicator	Condition	Signal Label	Score
RSI (14)	RSI < 25	Oversold (Strong)	+0.8
RSI (14)	25 <= RSI < 30	Oversold (Moderate)	+0.5
RSI (14)	30 <= RSI <= 70	Neutral	0.0
RSI (14)	70 < RSI <= 75	Overbought (Moderate)	-0.5
RSI (14)	RSI > 75	Overbought (Strong)	-0.8
MACD	Histogram > 0.5 and line > signal	Bullish Crossover (Strong)	+0.6
MACD	Histogram > 0 and line > signal	Bullish Crossover (Mod.)	+0.3
MACD	Histogram < -0.5 and line < signal	Bearish Crossover (Strong)	-0.6
MACD	Histogram < 0 and line < signal	Bearish Crossover (Mod.)	-0.3
Bollinger	Price < BB Lower	Below Lower Band	+0.6
Bollinger	Price within 15% of BB Lower	Near Lower Band	+0.4
Bollinger	Price within 15% of BB Upper	Near Upper Band	-0.4
Bollinger	Price > BB Upper	Above Upper Band	-0.6

Composite TA Score $= \frac{(\text{RSI_score} + \text{MACD_score} + \text{BB_score})}{3}$, clamped to [-1.0, +1.0].

3.3 News Sentiment Scoring

Headlines are fetched via `yfinance.news`, ticker-specific articles curated by Yahoo Finance, eliminating the need for keyword filtering. All headlines are scored in a single batched FinBERT forward pass:

$$\text{Sentiment_score} = \frac{\sum(\text{value x confidence})}{\sum(\text{confidence})}$$

where: positive = +1.0, negative = −1.0, neutral = 0.0.

The final label is assigned as: Positive (> +0.15), Negative (< −0.15), Neutral (between).

Model: `tahp0604/finbert-sentfin`, fine-tuned from `ProsusAI/finbert` on SEntFiN 1.0 (9,514 expert-annotated Indian financial headlines from Economic Times and Moneycontrol).

Training: Kaggle T4 GPU, 4 epochs, lr = 2e-5, fp16.

Best checkpoint: Epoch 3, **F1 weighted = 0.873** (vs ~0.76 baseline on Indian text).

3.4 Explainable Signal Fusion (ESF)

The core research contribution is that the fusion runs entirely in Python before LLM is called:

$$\text{fusion_score} = 0.60 \text{ x TA_score} + 0.40 \times \text{Sentiment_score}$$

fusion_score	conflict_detected	Overall Signal
> +0.15	False	Bullish
< -0.15	False	Bearish
-0.15 to +0.15	False	Neutral
Any	True	Conflicted

Conflict is flagged when TA and Sentiment have opposite signs AND |TA_score - Sentiment_score| > 0.3. Severity is High if difference > 0.6, else Moderate.

Key architectural point: The conflict flag is a deterministic Python result, not discovered by the LLM. The LLM receives it as a structured field in the prompt and cannot silently resolve or ignore it. The LLM's only role is language generation, not signal reasoning.

3.5 LLM Digest Generation

A single Groq API call (LLaMA 3.3-70B) per stock per day generates the plain-English digest. The prompt passes the pre-compute `fusion_narrative` and the user's goal, and constrains the LLM to, for example:

```
"""
You are a friendly, plain-English stock assistant helping a retail investor.

YOUR TASK:
Write a 3-5 sentence plain-English digest for this user.
Rules:
1. Start with today’s price move and clearly state the overall signal (Bullish/Bearish/Conflicted) early.
2. Include specific indicator values: RSI (with interpretation), MACD direction, and Bollinger Band position.
3. If signals conflict, briefly explain both sides (e.g., momentum vs overbought).
4. Give a direct recommendation aligned with the user’s goal (buy/sell/hold) with a concrete condition if needed (e.g., “wait for RSI < 50”).
5. Keep it 3–5 sentences, under 130 words, no vague language or internal metrics.
"""
```

4. Architecture

4.1 System Architecture Diagram

The architecture diagram below shows the two-path design:

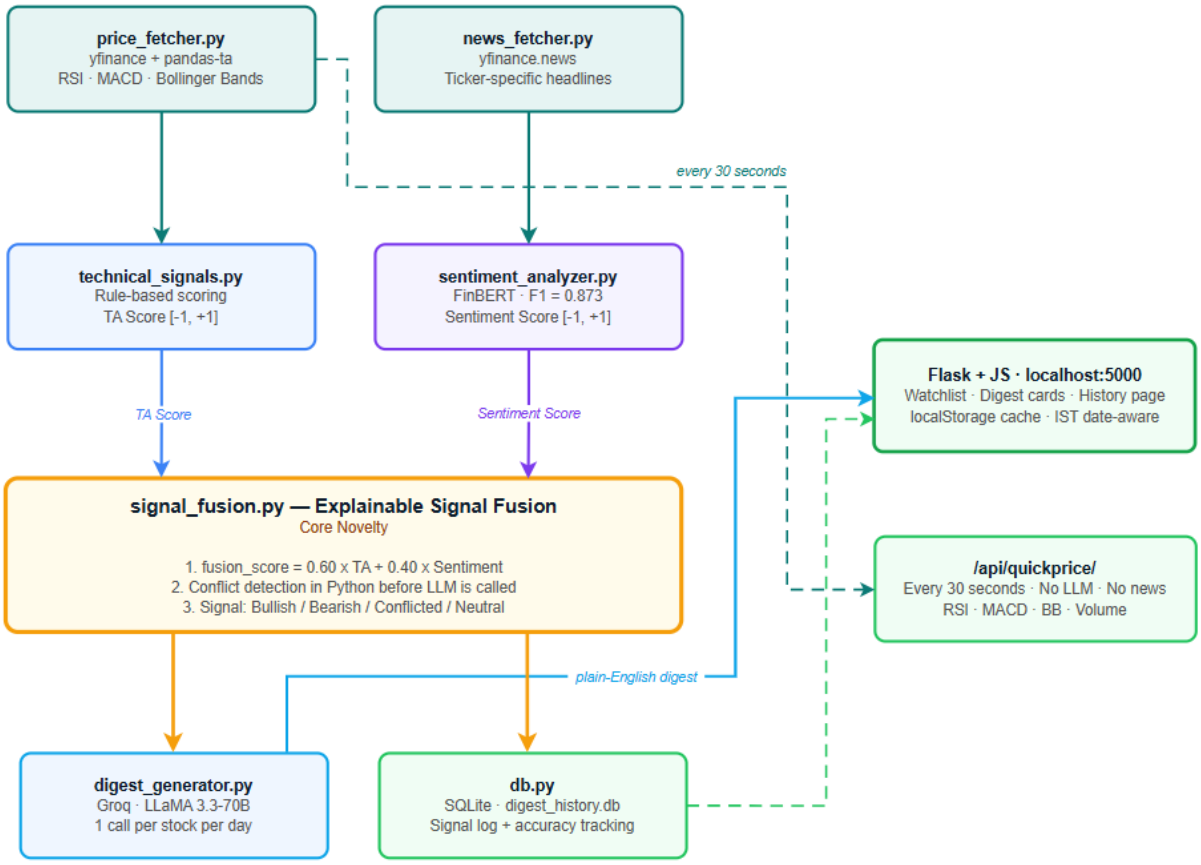

Figure 1: End-to-end pipeline for explainable stock signal generation combining technical indicators and news sentiment, served via a Flask dashboard and API.

4.2 Two-Path Design

This application runs two completely independent data flows per ticker simultaneously:

Path	Trigger	What It Does
Daily Digest	User clicks Run Analysis (once/day)	Full pipeline: price → news → FinBERT → ESF → Groq. Result cached in localStorage by ticker + IST date.
Live Block	Automatic every 30 seconds	/api/quickprice/<ticker>: yfinance + pandas-ta only. No LLM, no news. RSI, MACD, BB, Volume update in-place.

4.3 Project Structure

```
stock_watchlist/
   webapp/
     app.py                  Flask routes and API endpoints
     templates/
       index.html            Main watchlist page
       history.html          14-day insights page + further insights
     static/
       app.js                Frontend: live polling, cards, localStorage
       style.css             Styling
   src/
     ui/
       config.py             All constants, thresholds, model path
       models.py             Dataclasses: PriceData, FusionResult, DigestCard
       pipeline.py           Orchestrator
       db.py                 SQLite: save + query daily signal history
       data/
       price_fetcher.py      yfinance + pandas-ta: OHLCV + indicators
       news_fetcher.py       yfinance.news: ticker-specific headlines
     signals/
       technical_signals.py  Rule-based TA scoring
       sentiment_analyzer.py Fine-tuned FinBERT batched inference
     fusion/
       signal_fusion.py      ESF: weighted fusion + conflict detection
     llm/
       digest_generator.py   Groq prompt construction + API call
   scripts/scheduler.py      APScheduler daily job (optional)
   tests/                    Unit tests (no API keys needed)
   requirements.txt
   .env.example              Copy to .env and add GROQ_API_KEY
   setup.sh / setup.bat      One-command setup
   run.sh / run.bat          One-command run
   Dockerfile
   docker-compose.yml
```

4.4 API Endpoints

Method	Endpoint	Purpose	Called By
GET	/	Render main page	Browser on load
POST	/watchlist/add	Add ticker + goal	app.js addTicker()
DELETE	/watchlist/remove/<ticker>	Remove ticker	app.js removeTicker()
GET	/analyse/<ticker>	Full pipeline - returns DigestCard JSON	app.js fetchDigest()

Method	Endpoint	Purpose	Called By
GET	/api/quickprice/<ticker>	Fast price + indicators, no LLM	app.js every 30 seconds
GET	/api/pricehistory/<ticker>?period=	OHLCV + DB signals for history page	history.html on load
GET	/api/search?q=	Yahoo Finance ticker autocomplete	Input keystroke
GET	/history/<ticker>	Render history page	History link on card

4.5 Configuration Reference

All tuneable parameters are in `src/config.py`:

Parameter	Default	Effect
FINBERT_MODEL	Auto-detect	Local path or HuggingFace repo
GROQ_MODEL	llama-3.3-70b-versatile	LLM model for digest generation
WEIGHT_TECHNICAL	0.60	TA weight in fusion score (must sum to 1.0 with WEIGHT_SENTIMENT)
WEIGHT_SENTIMENT	0.40	Sentiment weight
CONFLICT_THRESHOLD	0.3	Minimum |TA - Sentiment| to flag a conflict
RSI_OVERSOLD	30.0	RSI below this = Oversold (Bullish signal)
RSI_OVERBOUGHT	70.0	RSI above this = Overbought (Bearish signal)
NEWS_MAX_HEADLINES	5	Headlines fetched per ticker per run
DIGEST_HOUR	15	Scheduled digest hour IST (scheduler.py only)

5. Usage and Scope

5.1 Adding Stocks to Your Watchlist

1. Open `http://localhost:5000` after starting the app.
2. Click the ticker input field and start typing a company name or symbol.
3. Select from the live dropdown (powered by Yahoo Finance autocomplete).
4. Enter a plain-English personal goal in the second field.
5. Click Add.

Exchange	Ticker Format	Examples
NSE (India)	TICKER.NS	RELIANCE.NS, TCS.NS, HDFCBANK.NS
BSE (India)	TICKER.BO	RELIANCE.BO, ADANIENSOL.BO
US Stocks	TICKER	AAPL, MSFT, TSLA, NVDA
ETFs	TICKER	SPY, QQQ, NIFTYBEES.NS

Goal examples: "tell me when to buy", "when should I sell", "is now a good long-term entry", "short term trade over 10 days", "I'm holding at a loss, should I cut or hold", "already invested, alert me if sentiment turns negative". More specific goals produce better digests.

5.2 Running Analysis

Click **"Run Today's Analysis"**. Skeleton cards appear immediately. Cards populate in parallel as each ticker finishes (~7 seconds for 3 stocks). The button only runs tickers without existing cards, existing cards are preserved.

5.3 Reading a Card

Each card has 3 sections:

5.3.1 Header

Displays company name, ticker symbol, and a signal badge: Signals in agreement or Conflicting signals (TA and sentiment pointing in opposite directions).

5.3.2 Live Block (updates every 30 seconds)

Field	What It Shows	How to Read
Price + Change %	Current close vs previous close	Green = up today, Red = down today
RSI (14)	Momentum indicator 0–100	< 30 = oversold (buy signal), > 70 = overbought (sell signal), 30–70 = neutral
MACD / Signal / Hist	Trend momentum	Histogram positive = bullish momentum, negative = bearish
BB Upper/Mid/Lower	Price range bands	Price above upper = overbought; below lower = oversold
Volume / Vol Avg 20d	Today vs 20-day average	Volume spike triangle = unusual activity

5.3.3 Today's Analysis Block (once daily)

- **Sentiment:** Positive / Negative / Neutral — from FinBERT analysis of recent news headlines.

- **Signal:** Bullish / Bearish / Neutral / Conflicted — the final fused signal from both TA and sentiment.
- **LLM Digest:** Plain-English paragraph with specific values, explanation of any conflict, and a direct recommendation tied to your goal.
- **Top Headline:** Most directionally significant news article found today.

5.4 14-Day History Panel

Click **"14-day history"** on any card to expand an inline panel showing:

- Price sparkline with signal dots (blue = Bullish, red = Bearish, yellow = Conflicted).
- Signal accuracy %: how often Bullish/Bearish predictions were correct the next day.
- Conflict dates where TA and sentiment disagreed.

Click **"Full insights"** for the detailed history page with period selector (2 weeks to 1 year), RSI chart, MACD histogram, conflict timeline, and full daily data table.

Note: Signal accuracy and the signal overlay only appear after running analysis on multiple consecutive days. On first use these sections show 'No signal history yet'.

5.5 Scope and Limitations

- yfinance price data has a ~15-minute delay relative to live market feeds.
- FinBERT covers financial English; performance may vary on non-English text.
- Signal accuracy is meaningful only after several days of consecutive analysis runs.
- The system does not place orders or connect to any brokerage.

6. Impact Overview Statement

6.1 Research Contribution: Explainable Signal Fusion

A systematic search across Google Scholar, arXiv, Semantic Scholar, ACM, IEEE, and Springer return zero results for the term "Explainable Signal Fusion" as of April 2026. The pattern is absent from all prior systems reviewed.

The key architectural distinction is where the fusion decision is made:

Aspect	Existing Systems	This project (ESF)
Signal fusion	Implicit inside LLM or neural net	Explicit Python code
Conflict detection	Not surfaced to user	Detected before LLM, logged to DB
LLM role	Fuses + explains simultaneously	Narrates only
Target user	Algo traders / researchers	Retail investors
Cost per stock/day	5–10 GPT-4 calls	1 Groq call (free tier)
Reproducible?	No	Unit-tested

6.2 Domain Contribution — Fine-tuned FinBERT

Metric	Value	Notes
F1 Weighted (validation)	0.873	Best checkpoint: Epoch 3 of 4
Improvement over baseline	+11%	vs ProsusAI/finbert on Indian text
Positive class F1	0.89	504 validation samples
Negative class F1	0.88	407 validation samples
Neutral class F1	0.86	517 validation samples
Training data	8,086 samples	SEntFiN 1.0, stratified 85/15 split
Dataset source	Economic Times + Moneycontrol	NSE500 companies, 2002–2017

Model hosted publicly at: `huggingface.co/tahp0604/finbert-sentfin`

6.3 Impact Summary Metrics

Metric	Value	Context
F1 on Indian financial news	0.873	vs ~0.76 ProsusAI/finbert baseline
Google Scholar hits for ESF	0	Term confirmed novel April 2026
Groq API calls per stock/day	1	vs 5–10 GPT-4 calls in competing systems
Parallel analysis (3 stocks)	~7 seconds	vs ~21 seconds sequential
API cost to operate daily	Free	Groq free tier + yfinance (no key)
Supported exchanges	NSE, BSE, US	Via Yahoo Finance search API

6.4 Closing Statement

This application proposal seeks to solve a very real and rapidly increasing problem in the Indian retail investment area. India has opened around 30 million new demat accounts in the year 2023-24. Most of those investors do not have any method to evaluate stock signals and merely depend on unverified tips. By providing institutional-grade multi-signal analysis at almost no cost, through the use of free API tiers, open-source models, and a fine-tuned domain-adapted FinBERT, this application proves that principled reproducible explainable AI can be made available to those who are most in need of it. The design of the architecture also has a bearing on regulatory compliance. SEBI and RBI are ramping up their scrutiny of AI-generated financial advice. The ESF architecture is capable of recording everything: each conflict detection, fusion score, and signal classification is a logged Python object with a deterministic and traceable computation path.

7. Troubleshooting

Error	Cause	Fix
GROQ_API_KEY is not set	Missing or empty .env	Copy .env.example to .env and add your Groq key
No price data for ticker	Wrong format or delisted stock	Use TICKER.NS for NSE, TICKER.BO for BSE, TICKER for US
MACD / Signal showing --	Insufficient history for new listings	app.py uses period='1y' for warmup; may still fail for stocks < 35 trading days old
Cards not loading after reload	Stale localStorage format	Open DevTools Console: localStorage.clear(); location.reload()

7.1 Verifying Raw Indicator Values

If card values look suspicious, run the verification script to compare against TradingView:

```
python src/verify_ticker.py TCS.NS
```

This prints raw RSI, MACD, and Bollinger Band values. Cross-check against TradingView's RSI (14) and MACD (12, 26, 9). Values should match within ±0.5 due to different history warmup lengths.

References

[1] Xiao et al. (2024). TradingAgents: Multi-Agents LLM Financial Trading Framework. arXiv:2412.20138.

[2] Fatouros et al. (2025). MarketSenseAI 2.0: Enhancing Stock Analysis through LLM Agents. arXiv:2502.00415.

[3] Tong et al. (2024). Ploutos: Towards Interpretable Stock Movement Prediction with Financial LLM. arXiv:2403.00782.

[4] Koa et al. (2024). Learning to Generate Explainable Stock Predictions using Self-Reflective LLMs. arXiv:2402.03659.

[5] Sinha et al. (2022). SEntFiN 1.0: Entity-Aware Sentiment Analysis for Financial News. JASIST, 73(9), 1314–1335.

[6] Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-Trained Language Models. arXiv:1908.10063.

[7] Zhang et al. (2024). FinAgent: A Multimodal Foundation Agent for Financial Trading. KDD 2024.